

A • Python 基础语法

数据类型 不可变: int, float, str, bool, tuple; **可变**: list, dict, set

字典键必须不可变且唯一。转换: int("3")-3, list("ab")-["a", "b"]

运算符 算术: + - * /(真除) //(整除)%(取余)**(幂)

比较: == != > < >= <=; 逻辑: and or not (短路求值)

成员: in, not in; 身份: is, is not

控制结构 if/elif/else 多分支; for x in range(n): 循环 0 到 n-1

for i,v in enumerate(L): 获取索引 + 值

while cond: 条件循环; break 退出; continue 跳过

列表推导: [x**2 for x in range(5) if x>1]→[4,9,16]

函数 def func(a, b=1): 默认参数在后; 无 return 返回 None

lambda x,y: x+y 匿名函数

内置: len(), range(start,stop,step), sorted(L, key=..., reverse=True),

zip(), map(), filter(), sum(), max(), min(), abs(), round()

字符串与列表 切片: s[1:4] 索引 1-3, s[::-1] 反转

字符串: .split(",") , ", ".join(L), .strip(), .replace(a,b), .upper(),

.lower(), .find("x")(-1= 未找到)

格式化: f"{val:.2f}" 保留 2 位小数

列表: .append(x), .pop(), .insert(i,x), .sort(), .reverse(), .index(x),

.remove(x)

字典: d[k]=v, .get(k, default), .keys(), .values(), .items() B-A={5,6}属于 B但不属于 A

集合: A={1,2,3,4}, B={3,4,5,6} 交 A&B={3,4} 并 A|B={1,2,3,4,5,6} 差 A-B={1,2} 属于 A 但不属于 B

面向对象 类: class 定义的数据与方法封装体; **对象**: 类的实例

__init__(self,...) 构造函数; **继承**: class Child(Parent): 复用父类

PyTorch 中: class Net(nn.Module): 必须继承 nn.Module, 实现 __init__ 和

forward

A • 排序算法 (5 种)

算法	最好	平均	最坏	空间	稳定
冒泡	n	n^2	n^2	1	是
选择	n^2	n^2	n^2	1	否
插入	n	n^2	n^2	1	是
快速	$n\log n$	$n\log n$	n^2	$\log n$	否
归并	$n\log n$	$n\log n$	$n\log n$	n	是

冒泡: 相邻比较交换, 每轮最大值冒泡到末尾。核心: if a[j]>a[j+1]: swap

选择: 每轮找最小值放到已排序末尾。min_idx=i; for j in range(i+1,n)

插入: 将当前元素插入已排序部分正确位置。类似整理扑克牌

快速: 选基准 pivot, 分 <pivot 和 >pivot 两部分, 递归。平均最快

归并: 分治法, 递归拆两半 → 分别排序 → 合并。稳定且保证 $O(n\log n)$

A • 查找算法

线性查找 $O(n)$ 逐个遍历, 不要求有序。

二分查找 $O(\log n)$ (2024 真题) 要求数组有序。每次排除一半:

low, high=0, len(arr)-1

while low<=high:

mid=(low+high)//2

if arr[mid]==target: return mid

elif arr[mid]<target: low=mid+1

else: high=mid-1

易错: <= 不是 <; mid+1/mid-1 不是 mid

递归 函数调用自身, **必须有终止条件**。

def gcd(a,b): return a if b==0 else gcd(b,a%b)

def fib(n): return n if n<=1 else fib(n-1)+fib(n-2)

B • 人工智能概论 (简答题点)

AI 定义与三大学派 **人工智能**: 使计算机模拟人类智能行为的技术, 包括学习、推

理、感知和决策能力。

连接主义: 模拟大脑神经网络, 调整连接权重学习。代表: 深度学习、Hinton。

符号主义: 用逻辑符号和规则表示知识, 推理解决问题。代表: 专家系统、Newell。

行为主义: 与环境交互, 通过奖励信号学习策略。代表: 强化学习。

图灵测试 Turing 1950 年提出: 人类评判者在文字对话中无法区分机器与人。

则认为机器具有智能。局限: 只测语言能力。中文房间论证 (Searle) 质疑其有效性。

AI 里程碑 1956 达特茅斯 →1997 深蓝 →2012 AlexNet→2016 AI-

phaGo→2017 Transformer→2022 ChatGPT

B • AI 伦理 (简答题必考)

数据隐私 个人对其信息的收集、存储、使用拥有知情权和控制权。GDPR (欧盟

2018) 确立数据最小化、知情同意原则。中国《个人信息保护法》2021 年实施。

算法偏见 AI 对某些群体产生系统性不公平对待。来源: (1) **数据偏见**——训练数

据不代表真实多样性 (如招聘 AI 歧视女性); (2) **设计偏见**——特征选择隐含不公平假设; (3) **交互偏见**——用户行为强化刻板印象。解决: 多样化数据、公平性措

标、人工审核。

责任归属 AI 出错时谁负责? 涉及开发者、使用者、数据提供者、监管方。核心

难题: 深度学习**黑箱特性**使决策难以追溯。

可解释性 AI 决策过程能被人类理解的程度。高风险场景 (医疗、司法、信贷) 必

须具备。欧盟 AI Act 要求高风险 AI 系统具备透明度。

B • 机器学习 (简答 + 选择重点)

监督学习: 有标签, 学输入 → 输出映射。分类 (离散) 和回归 (连续)。

例: 垃圾邮件、房价。

无监督学习: 无标签, 发现数据结构。聚类 (K-Means) 和降维 (PCA)。

强化学习: 智能体与环境交互, 奖励信号学最优策略。例: AlphaGo。核心: 状

态、动作、奖励、策略。

算法	类型	简答要点
线性回归	回归	$y=wx+b$, 最小化 MSE
逻辑回归	分类	Sigmoid 输出概率, 阈值 0.5
KNN	分类	K 个近邻投票, K 选奇数
决策树	分类	树状规则, 可解释性强
随机森林	集成	多棵决策树 + 投票
K-Means	聚类	迭代更新 K 个质心

混淆矩阵与评估指标 **标准形式** (行 = 实际, 列 = 预测):

	预测正 (P)	预测负 (N)
实际正	TP 真正例	FN 假负例
实际负	FP 假正例	TN 真负例

Accuracy 准确率 $\frac{TP+TN}{TP+TN+FP+FN}$

整体判断正确的比例。样本均衡时用。

Precision 精确率 $\frac{TP}{TP+FP}$

预测为正的里有多少真正。垃圾邮件过滤用。

Recall 召回率/灵敏度 $\frac{TP}{TP+FN}$

实际为正的里找回多少。疾病筛查用。

Specificity 特异度 $\frac{TN}{TN+FP}$

实际为负的里判对多少。排除健康人用。

F1-Score $\frac{2 \times P \times R}{P+R}$

精确率与召回率的调和平均。样本不均衡时用。

例: 100 样本 (50 正 50 负), TP=40, FP=10, FN=10, TN=40

Acc= $\frac{80}{100}$ =80%, Prec= $\frac{40}{50}$ =80%, Recall= $\frac{40}{50}$ =80%, Spec= $\frac{40}{50}$ =80%, F1= $\frac{2 \times 0.8 \times 0.8}{0.8+0.8}$ =80% **训练集**模型学习数据规律, **验证集**调整模型参数 (防止拟合), **测试集**最终评估模型性能。

过拟合 vs 欠拟合 **过拟合**: 训练好测试差——记住噪声。解决: 正则化 (L1/L2)、偏差-方差权衡: 欠拟合对应高偏差 (模型预测的期望值与真实值偏离大), 过拟合对应高方差 (模型在不同训练集上预测结果波动大)。一个好的模型需要在偏差和方差之间找到一个平衡点。

欠拟合: 训练测试都差——太简单。解决: 增复杂度、增特征、减正则化。

交叉验证: K 折, 轮流用 1 份验证、K-1 份训练。

集成学习 Bagging: 并行训练 + 投票 → **降方差**。随机森林 =Bagging+ 决策树。

Boosting: 串行纠错 → **降偏差**。代表: XGBoost。

B • 神经网络与深度学习

基本结构 **感知机**: 单层, 只能线性可分 (不能解 XOR)。

MLP: 输入 → 隐藏 (可多层) → 输出。隐藏层引入非线性 → 解 XOR。

正向传播: $z = Wx + b$ → 激活 → 输出。

反向传播: **链式法则**从输出向输入计算梯度, 训练核心算法。

函数	公式	输出	用途
Sigmoid	$\frac{1}{1+e^{-x}}$	(0, 1)	二分类输出
激活函数 ReLU	$\max(0, x)$	$[0, \infty)$	隐藏层首选
Tanh	$\frac{e^x - e^{-x}}{e^x + e^{-x}}$	(-1, 1)	中心化
Softmax	$\frac{e^{x_i}}{\sum e^{x_j}}$	概率分布	多分类输出

$\sigma' = \sigma(1-\sigma)$; ReLU 导数: $x>0$ 为 1, $x<0$ 为 0

损失函数与梯度下降 $MSE=\frac{1}{n} \sum (\hat{y} - y)^2$ → 回归; **交叉熵**

$=-\sum y_i \log \hat{y}_i$ → 分类

更新: $\theta \leftarrow \theta - \alpha \nabla L$; α 学习率过大 → 震荡; 过小 →

太慢SGD→Mini-batch→Adam (自适应, **最常用**)

B • CNN 卷积神经网络

卷积层: 卷积核滑动提取局部特征, 参数共享减少参数量。

池化层: 下采样减特征图尺寸。没有可学习的参数, MaxPool 取最大值, 平均池化取平均值。

输出尺寸 (必背): Out = $\lfloor \frac{In-K+2P}{S} \rfloor + 1$; 池化: Out = In/PoolSize; 展

平: $C \times H \times W$ **In**输入尺寸, **Out**输出尺寸, **K**核尺寸, **P**填充, **S**步长, **Pool**池化, **C**通道, **H/W**高/宽, **向下取整**例子 $\lfloor 39.5 \rfloor = 39$

LeNet-5 逐层结构(2024 真题)

层	操作	输出尺寸	计算说明
输入	—	$1 \times 28 \times 28$	灰度图像
Conv1	Conv2d(1,6,5)	$6 \times 24 \times 24$	$(28-5)+1=24$
Pool1	MaxPool(2,2)	$6 \times 12 \times 12$	$24 \div 2=12$
Conv2	Conv2d(6,16,5)	$16 \times 8 \times 8$	$(12-5)+1=8$
Pool2	MaxPool(2,2)	$16 \times 4 \times 4$	$8 \div 2=4$
Flatten	—	256	$16 \times 4 \times 4=256$
FC1	Linear(256,120)	120	全连接层
FC2	Linear(120,84)	84	全连接层
FC3	Linear(84,10)	10	输出类别数

B • LLM 与数据处理

LLM: 基于 Transformer, "预测下一个词"。代表: GPT、BERT、Llama。

Transformer(2017): 核心**自注意力**(Self-Attention), Q/K/V 计算词间关联。

预训练 → **微调**: 大数据学通用知识 → 小数据针对特定任务。

数据处理: 清洗 → 划分 (80/20 或 70/15/15) → 标准化/归一化。

特征工程: 选择、提取、构造有用特征。

数学 • 概率与统计

$P(A') = 1-P(A)$; 独立: $P(A \cap B) = P(A)P(B)$; 条件:

$P(A|B) = \frac{P(A \cap B)}{P(B)}$

贝叶斯: $P(A|B) = \frac{P(B|A)P(A)}{P(B)}$

例: 患病率 1%, 灵敏度 99%, 假阳率 5%。阳性时真患病?

$\frac{0.99 \times 0.01}{0.99 \times 0.01 + 0.05 \times 0.99} = \frac{0.0099}{0.0594} \approx 16.7\%$

$\mu = \frac{1}{n} \sum x_i$, $\sigma^2 = \frac{1}{n} \sum (x_i - \mu)^2$, 标准化 $z = (x - \mu) / \sigma$

正态分布 $X \sim N(\mu, \sigma^2)$: 钟形曲线, 关于 μ 对称, σ 控制宽窄。

68-95-99.7 规则:

$P(\mu-\sigma < X < \mu+\sigma) \approx 68.3\%$

$P(\mu-2\sigma < X < \mu+2\sigma) \approx 95.4\%$

$P(\mu-3\sigma < X < \mu+3\sigma) \approx 99.7\%$

标准正态 $Z \sim N(0,1)$: $z = (x - \mu) / \sigma$, 常用 CDF 值:

$\Phi(1)=0.841, \Phi(2)=0.977, \Phi(-z)=1-\Phi(z)$

CDF: $F(x)=P(X \leq x), P(a < X \leq b)=F(b)-F(a)$

例: 成绩 $\sim N(70, 10^2)$, $P(60 < X < 80)=P(-1 < Z < 1) \approx 68.3\%$

$P(X > 90)=P(Z > 2)=1-0.977=2.3\%$

二项: $P(X=k)=C_n^k p^k (1-p)^{n-k}, \mu=np, \sigma^2=np(1-p)$

均匀 $[a,b]$: $f=\frac{1}{b-a}, \mu=\frac{a+b}{2}, P(c < X < d)=\frac{d-c}{b-a}$

数学 · 导数、梯度与线性代数

$$\begin{matrix} x^n & nx^{n-1} \end{matrix} \bigg| \begin{matrix} e^x & e^x \end{matrix} \bigg| \begin{matrix} \ln x & 1/x \end{matrix}$$

$$\sigma(x) \quad \sigma(1-\sigma) \quad \bigg| \quad \text{ReLU} \quad \begin{cases} 0 \\ 1 \end{cases} \quad \bigg| \quad \begin{matrix} \text{常数} & 0 \end{matrix}$$

链式: $\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$; 梯度 = $(\frac{\partial L}{\partial w_1}, \dots)$ 指向增长最快方向

例: $L = (wx - y)^2, w = 2, x = 3, y = 5, \alpha = 0.1$

$\hat{y} = 6, L = 1, \frac{\partial L}{\partial w} = 2(6 - 5) \times 3 = 6, w_{\text{new}} = 2 - 0.6 = 1.4$

点积 $\mathbf{a} \cdot \mathbf{b} = \sum a_i b_i$; 矩阵 $(m \times k) \times (k \times n) = (m \times n)$, **前列 = 后行**
 $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 5 \\ 6 \end{pmatrix} = \begin{pmatrix} 17 \\ 39 \end{pmatrix}$; 转置 $(A^T)_{ij} = A_{ji}$

C · NumPy 速查

操作	代码
----	----

创建	np.array([1,2,3])
全零/一	np.zeros((m,n))/np.ones
形状	a.shape, a.reshape(m,n)
转置	a.T
矩阵乘	A @ B 或 np.dot(A,B)
逐元素	A*B, A+B, A**2
统计	.mean(), .std(), .sum(axis=)
布尔索引	a[a>0]
展平	a.flatten()
随机	np.random.rand(m,n)

A · 文件读写

with open("f.txt","r") as f: text=f.read()

with open("f.txt","w") as f: f.write("hi")

模式: "r" 读 "w" 写 (覆盖) "a" 追加 "rb" 二进制读

f.readlines() 返回列表; for line in f: 逐行读

A · 异常处理

try: 可能出错的代码

except ValueError as e: 捕获特定异常

except Exception as e: 捕获所有异常

finally: 无论是否异常都执行 (清理资源)

常见: ValueError, TypeError, IndexError, KeyError, ZeroDivisionError

A · 常用模块

import math: math.sqrt(), math.pi, math.ceil(), math.floor(), math.log()

import random: random.randint(a,b), random.random(), random.choice(L),

random.shuffle(L)

import os: os.path.exists(), os.listdir()

C · Pandas 速查

操作	代码
----	----

读取	pd.read_csv("file.csv")
查看	df.head(), df.info(), df.describe()
选列	df["col"] 或 df[["c1", "c2"]]
按标签/位置	df.loc[行, 列]/df.iloc[行号, 列号]
筛选	df[df["col"]>5]
缺失值	df.dropna()/df.fillna(v)
分组	df.groupby("c").mean()
合并	pd.merge(df1, df2, on="k")
排序	df.sort_values("c", ascending=F)

C · Matplotlib 速查

import matplotlib.pyplot as plt

plt.plot(x,y) 折线 / plt.bar(x,y) 柱状 / plt.scatter(x,y) 散点

plt.hist(data, bins=10) 直方图

plt.title/xlabel/ylabel("T"); plt.legend(); plt.show()

C · Scikit-Learn 速查与案例

标准流程 (必背)

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

1. 划分数据

```
X_train, X_test, y_train, y_test = \
    train_test_split(X, y, test_size=0.2,
                    random_state=42)
```

2. 标准化

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test) # 不用 fit!
```

3. 训练

```
model = LogisticRegression()
model.fit(X_train, y_train)
```

4. 预测与评估

```
y_pred = model.predict(X_test)
acc = accuracy_score(y_test, y_pred)
```

模型	类名
线性回归	LinearRegression()
逻辑回归	LogisticRegression()
KNN	KNeighborsClassifier(n=5)
随机森林	RandomForestClassifier()
SVM	SVC(kernel="rbf")

关键: 训练集 fit_transform, 测试集只 transform!

评估: accuracy_score, confusion_matrix, classification_report

数据预处理 MinMaxScaler(): 归一化到 [0,1]

LabelEncoder(): 标签编码 (字符串 → 数字)

OneHotEncoder(): 独热编码 (类别 → 向量)

train_test_split(X,y, test_size=0.2, stratify=y): 分层抽样

交叉验证

```
from sklearn.model_selection \
import cross_val_score
scores = cross_val_score(
    model, X, y, cv=5,
    scoring="accuracy")
print(f"Mean: {scores.mean():.3f}")
```

C · PyTorch 速查

方式一: 继承 nn.Module (灵活)

```
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1,6,5)
        self.pool = nn.MaxPool2d(2,2)
        self.conv2 = nn.Conv2d(6,16,5)
        self.fc1 = nn.Linear(16*4*4, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 16*4*4) # 展平
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        return self.fc3(x)
```

方式二: nn.Sequential (简洁, 无需写 forward)

```
model = nn.Sequential(
    nn.Conv2d(1, 6, 5), nn.ReLU(),
    nn.MaxPool2d(2, 2),
    nn.Conv2d(6, 16, 5), nn.ReLU(),
    nn.MaxPool2d(2, 2),
    nn.Flatten(), # 自动展平
    nn.Linear(16*4*4, 120), nn.ReLU(),
    nn.Linear(120, 84), nn.ReLU(),
    nn.Linear(84, 10))
```

按顺序自动执行, 适合简单顺序网络

训练循环 (必背)

```
optimizer = optim.Adam(
    model.parameters(), lr=0.001)
criterion = nn.CrossEntropyLoss()
```

```
for epoch in range(num_epochs):
    for X_batch, y_batch in dataloader:
        output = model(X_batch) # 正向
        loss = criterion(output, y_batch)
        optimizer.zero_grad() # 清零
        loss.backward() # 反向
        optimizer.step() # 更新
```

	层	参数
常用层速查	nn.Conv2d	(in_ch, out_ch, kernel)
	nn.MaxPool2d	(kernel, stride)
	nn.Linear	(in_feat, out_feat)
	nn.ReLU()	无参数
	nn.Flatten()	自动展平
	nn.Dropout(p)	丢弃概率 p
	nn.BatchNorm2d	(num_features)
	nn.Sequential	按顺序堆叠层

x.view(-1,n) 展平; 训练 model.train(); 评估 model.eval()

推理: with torch.no_grad(): 禁用梯度

回归 nn.MSELoss(); 分类 nn.CrossEntropyLoss()

Sequential vs Module: Sequential 适合顺序网络, Module 适合分支/跳连
数据加载

```
from torch.utils.data import \
    DataLoader, TensorDataset
dataset = TensorDataset(
    torch.tensor(X), torch.tensor(y))
loader = DataLoader(
    dataset, batch_size=32, shuffle=True)
```

Tensor 基本操作 torch.tensor([1,2,3]) 创建; .shape 形状

.to("cuda") 转 GPU; .numpy() 转 NumPy

torch.randn(m,n) 随机; torch.zeros/ones

.unsqueeze(dim)/.squeeze() 增/减维度

考试答题技巧

时间分配 (120 分钟): 单选 30min→ 简答 25min→ 补全 25min→ 编程 30min→

检查 10min

简答题: 先写定义 → 再写特点/区别 → 最后举例。不要只写关键词, 要写完整句子。

程序补全: 先通读全部代码理解整体逻辑 → 再逐空填写 → 注意缩进和冒号。

编程题: 先写注释规划思路 → 写核心逻辑 → 处理边界情况 → 即使不完整也要写部分代码拿分。

常见陷阱: = 赋值 vs== 比较; range(n) 是 0 到 n-1; 列表索引从 0 开始; // 整除 vs/ 真除; 缩进错误导致逻辑错误。